Table of Contents

Week 8: JDBC - Connecting Java to PostgreSQL

- Teaching Guide & Hands-on Project
- 📋 Session Overview
- 🎯 Learning Objectives
- 📚 Quick Review (5 minutes)
- 🕐 Detailed Lesson Plan (120 Minutes)
  - Part 1: Introduction to JDBC (10 minutes)
  - Part 2: Setting Up JDBC Driver (10 minutes)
  - Part 3: Database Configuration Class (10 minutes)
  - Part 4: Executing SQL Queries - SELECT (15 minutes)
- 💻 HANDS-ON PROJECT: Complete Student Management System with JDBC (50 minutes)
  - Project Overview
  - Project Structure
  - Task 1: Complete the DAO Implementation (20 minutes)
  - Task 2: Build the Main Application (30 minutes)
- 📝 Homework Assignment
  - Task 1: Add Course Management (Required)
  - Task 2: Implement Search Enhancement (Required)
  - Task 3: Add Transaction Management (Challenge)
- 🎯 Assessment Checklist
- 🔧 Common Student Errors & Solutions
  - Error 1: ClassNotFoundException
  - Error 2: Connection Refused
  - Error 3: Authentication Failed
  - Error 4: Database Does Not Exist
  - Error 5: Wrong Parameter Index
  - Error 6: Resource Leak
  - Error 7: Wrong Execute Method
- 📚 Additional Resources
  - Official Documentation:
  - Best Practices:
  - Video Tutorials:

🔜 Preview of Week 8

📌 Key Takeaways

🎓 Week 7 Summary

Part 6: Data Access Object (DAO) Pattern (10 minutes)

Week 8: JDBC - Connecting Java to PostgreSQL

Teaching Guide & Hands-on Project

📋 Session Overview

Duration: 2 - 3 Hours
Teaching: 70 minutes | **Hands-on Practice:** 50 minutes

🎯 Learning Objectives

By the end of this session, students will be able to:
1. Understand JDBC architecture and components
2. Set up PostgreSQL JDBC driver in Java projects
3. Establish database connections from Java
4. Execute SQL queries using Statement and PreparedStatement
5. Process query results with ResultSet
6. Implement CRUD operations (Create, Read, Update, Delete)
7. Prevent SQL injection attacks
8. Handle database exceptions properly
9. Manage database connections efficiently
10. Build a complete data access layer using DAO pattern

📚 Quick Review (5 minutes)

Quick Questions:
- What is a primary key?
- What is a foreign key?
- What's the difference between INNER JOIN and LEFT JOIN?
- How do you filter results in SQL?
- Why should you always use WHERE with UPDATE/DELETE?

Key Review Points:
- Databases store structured data in tables
- SQL is the language for querying databases

- Relationships are created with foreign keys
- Always use WHERE clause with UPDATE/DELETE

🕐 Detailed Lesson Plan (120 Minutes)

Part 1: Introduction to JDBC (10 minutes)

What is JDBC? (5 minutes)

Definition:
- **JDBC** = Java Database Connectivity
- Standard Java API for connecting to relational databases
- Allows Java applications to interact with databases
- Database-independent (works with PostgreSQL, MySQL, Oracle, etc.)

JDBC Architecture:

```
┌─────────────────────────────────────┐
│        Java Application             │
│  (Your Student Management System)   │
└─────────────────────────────────────┘
                │
                ↓
┌─────────────────────────────────────┐
│           JDBC API                  │
│  (java.sql.* classes)               │
└─────────────────────────────────────┘
                │
                ↓
┌─────────────────────────────────────┐
│        JDBC Driver                  │
│  (PostgreSQL Driver - postgresql.jar)│
└─────────────────────────────────────┘
                │
                ↓
┌─────────────────────────────────────┐
│     PostgreSQL Database             │
│  (Your university_db)               │
└─────────────────────────────────────┘
```

Key JDBC Components:
- **DriverManager** - Manages database drivers and connections
- **Connection** - Represents a connection to the database
- **Statement/PreparedStatement** - Executes SQL queries

- **DriverManager** - Manages database drivers and connections
- **Connection** - Represents a connection to the database
- **Statement/PreparedStatement** - Executes SQL queries
- **ResultSet** - Contains query results (rows returned from SELECT)
- **SQLException** - Handles database errors

Why Use JDBC? (5 minutes)

Before JDBC (File Storage):

```
❌ No query capability (must read entire file)
❌ Slow search operations (O(n) time)
❌ No concurrent access
❌ Manual data parsing
❌ No data integrity
❌ No relationships between entities
```

With JDBC (Database):

```
✅ Powerful SQL queries
✅ Fast searches with indexes (O(log n) time)
✅ Multiple users simultaneously
✅ Automatic data handling
✅ Data validation and constraints
✅ Transaction support (ACID properties)
✅ Relationships between tables
```

Part 2: Setting Up JDBC Driver (10 minutes)

Step 1: Download PostgreSQL JDBC Driver (3 minutes)

Two Ways to Add Driver:

Method 1: Maven (Recommended)

```xml
<!— Add to pom.xml —>
<dependencies>
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <version>42.7.1</version>
    </dependency>
</dependencies>
```

Method 2: Manual JAR

1. Download from: https://jdbc.postgresql.org/download/
2. Download latest .jar file (e.g., postgresql-42.7.1.jar)
3. In IntelliJ IDEA:
- File → Project Structure → Libraries
- Click + → Java
- Select downloaded JAR file
4. Click OK to add to project

Step 2: Verify Setup (7 minutes)

Test Connection Program:

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class TestConnection {
    public static void main(String[] args) {
        // Database connection details
        String url = "jdbc:postgresql://localhost:5432/university_db";
        String username = "postgres";
        String password = "your_password";  // Replace with your PostgreSQL password

        Connection connection = null;

        try {
            // Attempt to connect to database
            System.out.println("Connecting to database...");
            connection = DriverManager.getConnection(url, username, password);
            System.out.println("✓ Successfully connected to PostgreSQL database!");
            System.out.println("✓ Database: university_db");

        } catch (SQLException e) {
            System.out.println("✗ Connection failed!");
            System.out.println("Error: " + e.getMessage());
            e.printStackTrace();
        } finally {
            // Always close the connection
            if (connection != null) {
                try {
                    connection.close();
                    System.out.println("✓ Connection closed.");
                } catch (SQLException e) {
                    System.out.println("Error closing connection: " + e.getMessage());
                }
            }
        }
```

```
    }   }
```

Connection String Breakdown:

```
jdbc:postgresql://localhost:5432/university_db
  ↑       ↑           ↑        ↑         ↑
  |       |           |        |         |
Protocol  DBMS      Host     Port    Database Name
```

Common Connection Issues:

Error	Cause	Solution
"No suitable driver"	JDBC JAR not added	Add postgresql.jar to project
"Connection refused"	PostgreSQL not running	Start PostgreSQL service
"Password authentication failed"	Wrong password	Check PostgreSQL password
"Database does not exist"	DB not created	Create database in pgAdmin
"Port 5432 in use"	Wrong port	Check PostgreSQL port in pgAdmin

---

Part 3: Database Configuration Class (10 minutes)

Creating a Reusable Config (10 minutes)

Best Practice: Centralize Database Configuration

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class DatabaseConfig {
    // Database credentials (centralized)
    private static final String URL = "jdbc:postgresql://localhost:5432/university_db";
    private static final String USERNAME = "postgres";
    private static final String PASSWORD = "your_password";  // Change this!

    /**
     * Get a connection to the database
```

```
     * @return Connection object     * @throws SQLException if connection fails     */
    public static Connection getConnection() throws SQLException {        try {
            Connection connection = DriverManager.getConnection(URL, USERNAME, PASSWORD);
            System.out.println("✓ Database connection established.");
            return connection;        } catch (SQLException e) {
            System.out.println("✗ Failed to connect to database!");
            System.out.println("Error: " + e.getMessage());
            throw e;  // Re-throw to let caller handle        }    }                /**
     * Close a connection safely     * @param connection Connection to close     */
    public static void closeConnection(Connection connection) {
        if (connection ≠ null) {            try {                connection.close();
                System.out.println("✓ Connection closed.");
            } catch (SQLException e) {
                System.out.println("Error closing connection: " + e.getMessage());
            }        }    }                /**
     * Close all resources safely (connection, statement, resultset)
     * @param conn Connection to close     * @param stmt Statement to close
     * @param rs ResultSet to close     */
    public static void closeResources(Connection conn, Statement stmt, ResultSet rs) {
        // Close ResultSet        if (rs ≠ null) {            try {
                rs.close();            } catch (SQLException e) {
                System.out.println("Error closing ResultSet: " + e.getMessage());            }
        }                // Close Statement        if (stmt ≠ null) {            try {
                stmt.close();            } catch (SQLException e) {
                System.out.println("Error closing Statement: " + e.getMessage());            }
        }                // Close Connection        if (conn ≠ null) {            try {
                conn.close();            } catch (SQLException e) {
                System.out.println("Error closing Connection: " + e.getMessage());
            }        }    }                /**       * Test the database connection     */
    public static void testConnection() {        try {
            Connection conn = getConnection();
            System.out.println("✓ Database connection test successful!");
            closeConnection(conn);        } catch (SQLException e) {
            System.out.println("✗ Database connection test failed!");        }    }
             public static void main(String[] args) {        // Test the connection
        testConnection();    }    }
```

Why Use a Config Class?

- ✅ Centralized configuration (change in one place)
- ✅ Reusable across all database operations
- ✅ Consistent error handling
- ✅ Easy to maintain
- ✅ Supports best practices (resource cleanup)

Part 4: Executing SQL Queries - SELECT (15 minutes)

Statement vs PreparedStatement (5 minutes)

Statement - Simple but Dangerous:

```
Statement stmt = connection.createStatement();
String userInput = "Alice'; DROP TABLE students; --";
String sql = "SELECT * FROM students WHERE name = '" + userInput + "'";
// ⚠️ SQL INJECTION VULNERABILITY!
// This could execute: SELECT * FROM students WHERE name = 'Alice'; DROP TABLE students; --'
```

PreparedStatement - Safe and Recommended:

```
String sql = "SELECT * FROM students WHERE name = ?";
PreparedStatement pstmt = connection.prepareStatement(sql);
pstmt.setString(1, userInput);  // Parameters are safely escaped
// ✅ SAFE - No SQL injection possible
```

Always use PreparedStatement!

Comparison Table:

Feature	Statement	PreparedStatement
SQL Injection Safe	❌ No	✅ Yes
Precompiled	❌ No	✅ Yes
Performance (repeated queries)	Slower	Faster
Parameterized Queries	❌ No	✅ Yes
Recommended	❌ Never	✅ Always

SELECT Queries with ResultSet (10 minutes)

Example 1: Reading All Students

```
import java.sql.*;

public class SelectAllStudents {
    public static void main(String[] args) {
        Connection connection = null;
        PreparedStatement pstmt = null;
        ResultSet rs = null;
```

```
            // 1. Get database connection
            connection = DatabaseConfig.getConnection();
            // 2. Prepare SQL query
            String sql = "SELECT student_id, name, age, email, major, gpa FROM students ORDER
        BY name";
            pstmt = connection.prepareStatement(sql);
            // 3. Execute query and get results            rs = pstmt.executeQuery();
                        // 4. Process the ResultSet
            System.out.println("═══ All Students ═══");
            System.out.printf("%-10s %-20s %-5s %-25s %-20s %-5s\n",
                "ID", "Name", "Age", "Email", "Major", "GPA");
            System.out.println("-".repeat(90));                        int count = 0;
            while (rs.next()) {                // Get data from current row
                String id = rs.getString("student_id");
                String name = rs.getString("name");                int age = rs.getInt("age");
                String email = rs.getString("email");
                String major = rs.getString("major");
                double gpa = rs.getDouble("gpa");
                // Display the data
                System.out.printf("%-10s %-20s %-5d %-25s %-20s %.2f\n",
                    id, name, age, email, major, gpa);                count++;            }
                        System.out.println("-".repeat(90));
            System.out.println("Total students: " + count);
        } catch (SQLException e) {            System.out.println("✗ Error executing query!");
            System.out.println("Error: " + e.getMessage());            e.printStackTrace();
        } finally {            // 5. Always close resources (important!)
            DatabaseConfig.closeResources(connection, pstmt, rs);        }    }    }
```

Key ResultSet Methods:

```
// Getting data by column name
String name = rs.getString("name");
int age = rs.getInt("age");
double gpa = rs.getDouble("gpa");
boolean active = rs.getBoolean("is_active");
Date enrollDate = rs.getDate("enrollment_date");

// Getting data by column index (starts at 1, not 0!)
String name = rs.getString(1);
int age = rs.getInt(2);

// Navigation
boolean hasData = rs.next();  // Move to next row, returns false if no more rows
boolean isFirst = rs.isFirst();  // Check if on first row
boolean isLast = rs.isLast();    // Check if on last row
```

Example 2: SELECT with Parameters

```java
public class SelectWithParameters {

    // Find student by ID
    public static void findStudentById(String studentId) {
        Connection connection = null;
        PreparedStatement pstmt = null;
        ResultSet rs = null;

        try {
            connection = DatabaseConfig.getConnection();

            // SQL with parameter placeholder (?)
            String sql = "SELECT * FROM students WHERE student_id = ?";
            pstmt = connection.prepareStatement(sql);

            // Set parameter value (index starts at 1)
            pstmt.setString(5, student.getMajor());
            pstmt.setDouble(6, student.getGpa());

            return pstmt.executeUpdate() > 0;

        } catch (SQLException e) {
            System.out.println("Error adding student: " + e.getMessage());
            return false;
        } finally {
            DatabaseConfig.closeResources(conn, pstmt, null);
        }
    }

    @Override
    public Student getStudentById(String studentId) {
        Connection conn = null;
        PreparedStatement pstmt = null;
        ResultSet rs = null;

        try {
            conn = DatabaseConfig.getConnection();
            String sql = "SELECT * FROM students WHERE student_id = ?";
            pstmt = conn.prepareStatement(sql);
            pstmt.setString(1, studentId);
            rs = pstmt.executeQuery();

            if (rs.next()) {
                return extractStudentFromResultSet(rs);
            }

        } catch (SQLException e) {
            System.out.println("Error getting student: " + e.getMessage());
        } finally {
            DatabaseConfig.closeResources(conn, pstmt, rs);
```

```
        }                 return null;    }                @Override
    public List<Student> getAllStudents() {        List<Student> students = new ArrayList<>();
        Connection conn = null;        PreparedStatement pstmt = null;
        ResultSet rs = null;                 try {
            conn = DatabaseConfig.getConnection();
            String sql = "SELECT * FROM students ORDER BY name";
            pstmt = conn.prepareStatement(sql);            rs = pstmt.executeQuery();
                        while (rs.next()) {
                students.add(extractStudentFromResultSet(rs));            }
        } catch (SQLException e) {
            System.out.println("Error getting students: " + e.getMessage());
        } finally {            DatabaseConfig.closeResources(conn, pstmt, rs);        }
                return students;    }
    // Helper method to extract Student object from ResultSet
    private Student extractStudentFromResultSet(ResultSet rs) throws SQLException {
        return new Student(            rs.getString("student_id"),
            rs.getString("name"),            rs.getInt("age"),
            rs.getString("email"),            rs.getString("major"),
            rs.getDouble("gpa")        );    }
    // Other methods will be implemented in hands-on project...    @Override
    public List<Student> getStudentsByMajor(String major) {
        // Students will implement this        return new ArrayList<>();    }
    @Override    public List<Student> getHonorRollStudents(double minGpa) {
        // Students will implement this        return new ArrayList<>();    }
    @Override    public boolean updateStudent(Student student) {
        // Students will implement this        return false;    }                @Override
    public boolean updateGPA(String studentId, double newGpa) {
        // Students will implement this        return false;    }                @Override
    public boolean deleteStudent(String studentId) {        // Students will implement this
        return false;    }                @Override    public int getStudentCount() {
        // Students will implement this        return 0;    }                @Override
    public double getAverageGPA() {        // Students will implement this        return 0.0;
    }    }
```

💻 HANDS-ON PROJECT: Complete Student Management System with JDBC (50 minutes)

Project Overview

Build a complete console-based Student Management System that:

1. Connects to PostgreSQL database (Week 6's university_db)
2. Implements full CRUD operations using DAO pattern
3. Provides user-friendly menu interface
4. Handles all exceptions properly
5. Closes all resources correctly

Project Structure

```
StudentManagementJDBC/
├── DatabaseConfig.java          (Database connection)
├── Student.java                 (Model class)
├── StudentDAO.java              (Interface)
├── StudentDAOImpl.java          (Implementation)
└── StudentManagementApp.java   (Main application)
```

Task 1: Complete the DAO Implementation (20 minutes)

Students should complete these methods in StudentDAOImpl:

```java
@Override
public List<Student> getStudentsByMajor(String major) {
    List<Student> students = new ArrayList<>();
    Connection conn = null;
    PreparedStatement pstmt = null;
    ResultSet rs = null;

    try {
        conn = DatabaseConfig.getConnection();
        String sql = "SELECT * FROM students WHERE major = ? ORDER BY gpa DESC";
        pstmt = conn.prepareStatement(sql);
        pstmt.setString(1, major);
        rs = pstmt.executeQuery();

        while (rs.next()) {
            students.add(extractStudentFromResultSet(rs));
        }

    } catch (SQLException e) {
        System.out.println("Error: " + e.getMessage());
    } finally {
        DatabaseConfig.closeResources(conn, pstmt, rs);
    }

    return students;
}

@Override
public List<Student> getHonorRollStudents(double minGpa) {
    List<Student> students = new ArrayList<>();
    Connection conn = null;
```

```
    PreparedStatement pstmt = null;    ResultSet rs = null;            try {
        conn = DatabaseConfig.getConnection();
        String sql = "SELECT * FROM students WHERE gpa ≥ ? ORDER BY gpa DESC";
        pstmt = conn.prepareStatement(sql);        pstmt.setDouble(1, minGpa);
        rs = pstmt.executeQuery();                 while (rs.next()) {
            students.add(extractStudentFromResultSet(rs));        }
    } catch (SQLException e) {        System.out.println("Error: " + e.getMessage());
    } finally {        DatabaseConfig.closeResources(conn, pstmt, rs);    }
    return students;}                @Overridepublic boolean updateStudent(Student student) {
    Connection conn = null;    PreparedStatement pstmt = null;            try {
        conn = DatabaseConfig.getConnection();
        String sql = "UPDATE students SET name = ?, age = ?, email = ?, " +
                     "major = ?, gpa = ? WHERE student_id = ?";
        pstmt = conn.prepareStatement(sql);        pstmt.setString(1, student.getName());
        pstmt.setInt(2, student.getAge());        pstmt.setString(3, student.getEmail());
        pstmt.setString(4, student.getMajor());        pstmt.setDouble(5, student.getGpa());
        pstmt.setString(6, student.getStudentId());
        return pstmt.executeUpdate() > 0;             } catch (SQLException e) {
        System.out.println("Error updating student: " + e.getMessage());        return false;
    } finally {        DatabaseConfig.closeResources(conn, pstmt, null);    }    }
        @Overridepublic boolean updateGPA(String studentId, double newGpa) {
    Connection conn = null;    PreparedStatement pstmt = null;            try {
        conn = DatabaseConfig.getConnection();
        String sql = "UPDATE students SET gpa = ? WHERE student_id = ?";
        pstmt = conn.prepareStatement(sql);        pstmt.setDouble(1, newGpa);
        pstmt.setString(2, studentId);                return pstmt.executeUpdate() > 0;
            } catch (SQLException e) {
        System.out.println("Error updating GPA: " + e.getMessage());        return false;
    } finally {        DatabaseConfig.closeResources(conn, pstmt, null);    }    }
        @Overridepublic boolean deleteStudent(String studentId) {    Connection conn = null;
    PreparedStatement pstmt = null;            try {
        conn = DatabaseConfig.getConnection();
        String sql = "DELETE FROM students WHERE student_id = ?";
        pstmt = conn.prepareStatement(sql);        pstmt.setString(1, studentId);
        return pstmt.executeUpdate() > 0;             } catch (SQLException e) {
        System.out.println("Error deleting student: " + e.getMessage());        return false;
    } finally {        DatabaseConfig.closeResources(conn, pstmt, null);    }    }
        @Overridepublic int getStudentCount() {    Connection conn = null;
    PreparedStatement pstmt = null;    ResultSet rs = null;            try {
        conn = DatabaseConfig.getConnection();
        String sql = "SELECT COUNT(*) as count FROM students";
        pstmt = conn.prepareStatement(sql);        rs = pstmt.executeQuery();
        if (rs.next()) {            return rs.getInt("count");        }
    } catch (SQLException e) {        System.out.println("Error: " + e.getMessage());
    } finally {        DatabaseConfig.closeResources(conn, pstmt, rs);    }
    return 0;}                @Overridepublic double getAverageGPA() {
    Connection conn = null;    PreparedStatement pstmt = null;    ResultSet rs = null;
            try {        conn = DatabaseConfig.getConnection();
        String sql = "SELECT AVG(gpa) as avg_gpa FROM students WHERE gpa IS NOT NULL";
        pstmt = conn.prepareStatement(sql);        rs = pstmt.executeQuery();
```

```
        if (rs.next()) {            return rs.getDouble("avg_gpa");        }
    } catch (SQLException e) {        System.out.println("Error: " + e.getMessage());
    } finally {        DatabaseConfig.closeResources(conn, pstmt, rs);    }
    return 0.0;}
```

Task 2: Build the Main Application (30 minutes)

Complete StudentManagementApp.java:

```
import java.util.List;
import java.util.Scanner;

public class StudentManagementApp {
    private StudentDAO studentDAO;
    private Scanner scanner;

    public StudentManagementApp() {
        this.studentDAO = new StudentDAOImpl();
        this.scanner = new Scanner(System.in);
    }

    public void displayMenu() {
        System.out.println("\n╔═══════════════════════════════════════╗");
        System.out.println("│   STUDENT MANAGEMENT SYSTEM (JDBC)    │");
        System.out.println("╚═══════════════════════════════════════╝");
        System.out.println("1.  Add Student");
        System.out.println("2.  View All Students");
        System.out.println("3.  Find Student by ID");
        System.out.println("4.  Find Students by Major");
        System.out.println("5.  View Honor Roll (GPA ≥ 3.5)");
        System.out.println("6.  Update Student GPA");
        System.out.println("7.  Update Student Information");
        System.out.println("8.  Delete Student");
        System.out.println("9.  View Statistics");
        System.out.println("10. Exit");
        System.out.println("─────────────────────────────────────");
        System.out.print("Enter choice (1-10): ");
    }

    public void addStudent() {
        System.out.println("\n--- Add New Student ---");

        try {
            System.out.print("Student ID: ");
            String id = scanner.nextLine();

            System.out.print("Name: ");
            String name = scanner.nextLine();
```

```
                    System.out.print("Age: ");            int age = scanner.nextInt();
        scanner.nextLine();                        System.out.print("Email: ");
        String email = scanner.nextLine();
        System.out.print("Major: ");            String major = scanner.nextLine();
                    System.out.print("GPA (0.0-4.0): ");
        double gpa = scanner.nextDouble();            scanner.nextLine();
        Student student = new Student(id, name, age, email, major, gpa);
        if (studentDAO.addStudent(student)) {
            System.out.println("✓ Student added successfully!");            } else {
            System.out.println("✗ Failed to add student.");            }
    } catch (Exception e) {
        System.out.println("✗ Invalid input! " + e.getMessage());
        scanner.nextLine();        }    }                public void viewAllStudents() {
    System.out.println("\n--- All Students ---");
    List<Student> students = studentDAO.getAllStudents();
    if (students.isEmpty()) {            System.out.println("No students found.");
    } else {            System.out.printf("%-10s %-20s %-5s %-25s %-20s %-5s\n",
            "ID", "Name", "Age", "Email", "Major", "GPA");
        System.out.println("-".repeat(95));
        for (Student student : students) {
            System.out.printf("%-10s %-20s %-5d %-25s %-20s %.2f\n",
                student.getStudentId(),                    student.getName(),
                student.getAge(),                    student.getEmail(),
                student.getMajor(),                    student.getGpa());            }
        System.out.println("-".repeat(95));
        System.out.println("Total students: " + students.size());        }    }
         public void findStudentById() {
    System.out.println("\n--- Find Student by ID ---");
    System.out.print("Enter Student ID: ");        String id = scanner.nextLine();
             Student student = studentDAO.getStudentById(id);
    if (student ≠ null) {            System.out.println("\n✓ Student Found:");
        displayStudentDetails(student);        } else {
        System.out.println("✗ Student not found.");        }    }
public void findStudentsByMajor() {
    System.out.println("\n--- Find Students by Major ---");
    System.out.print("Enter Major: ");        String major = scanner.nextLine();
    List<Student> students = studentDAO.getStudentsByMajor(major);
    if (students.isEmpty()) {
        System.out.println("No students found in " + major);        } else {
        System.out.println("\n" + major + " Students:");
        for (int i = 0; i < students.size(); i++) {
            System.out.printf("%d. %s (GPA: %.2f)\n",
                i + 1, students.get(i).getName(), students.get(i).getGpa());            }
        System.out.println("\nTotal: " + students.size());        }    }
public void viewHonorRoll() {
    System.out.println("\n--- Honor Roll (GPA ≥ 3.5) ---");
    List<Student> honorStudents = studentDAO.getHonorRollStudents(3.5);
    if (honorStudents.isEmpty()) {
        System.out.println("No students on honor roll.");        } else {
        for (int i = 0; i < honorStudents.size(); i++) {
```

```
            Student s = honorStudents.get(i);
            System.out.printf("%d. %s (%s) - GPA: %.2f\n",
                i + 1, s.getName(), s.getMajor(), s.getGpa());            }
        System.out.println("\nTotal: " + honorStudents.size());        }    }
    public void updateGPA() {        System.out.println("\n--- Update Student GPA ---");
        System.out.print("Enter Student ID: ");        String id = scanner.nextLine();
            Student student = studentDAO.getStudentById(id);
        if (student == null) {            System.out.println("✗ Student not found.");
            return;        }
        System.out.println("Current GPA: " + student.getGpa());
        System.out.print("Enter new GPA (0.0-4.0): ");                try {
            double newGpa = scanner.nextDouble();            scanner.nextLine();
            if (newGpa < 0.0 || newGpa > 4.0) {
                System.out.println("✗ GPA must be between 0.0 and 4.0");
                return;            }
            if (studentDAO.updateGPA(id, newGpa)) {
                System.out.println("✓ GPA updated successfully!");            } else {
                System.out.println("✗ Failed to update GPA.");            }
        } catch (Exception e) {            System.out.println("✗ Invalid input!");
            scanner.nextLine();        }    }                public void updateStudentInfo() {
        System.out.println("\n--- Update Student Information ---");
        System.out.print("Enter Student ID: ");        String id = scanner.nextLine();
            Student student = studentDAO.getStudentById(id);
        if (student == null) {            System.out.println("✗ Student not found.");
            return;        }                System.out.println("\nCurrent Information:");
        displayStudentDetails(student);
        System.out.println("\nEnter new information (press Enter to keep current):");
        System.out.print("Name [" + student.getName() + "]: ");
        String name = scanner.nextLine();        if (!name.isEmpty()) student.setName(name);
            System.out.print("Age [" + student.getAge() + "]: ");
        String ageStr = scanner.nextLine();        if (!ageStr.isEmpty()) {            try {
            student.setAge(Integer.parseInt(ageStr));
            } catch (NumberFormatException e) {
            System.out.println("Invalid age, keeping current value.");            }
        }                System.out.print("Email [" + student.getEmail() + "]: ");
        String email = scanner.nextLine();
        if (!email.isEmpty()) student.setEmail(email);
        System.out.print("Major [" + student.getMajor() + "]: ");
        String major = scanner.nextLine();
        if (!major.isEmpty()) student.setMajor(major);
        System.out.print("GPA [" + student.getGpa() + "]: ");
        String gpaStr = scanner.nextLine();        if (!gpaStr.isEmpty()) {            try {
            student.setGpa(Double.parseDouble(gpaStr));
            } catch (NumberFormatException e) {
            System.out.println("Invalid GPA, keeping current value.");            }
        }                if (studentDAO.updateStudent(student)) {
            System.out.println("✓ Student information updated successfully!");        } else {
            System.out.println("✗ Failed to update student.");        }    }
    public void deleteStudent() {        System.out.println("\n--- Delete Student ---");
        System.out.print("Enter Student ID: ");        String id = scanner.nextLine();
```

```
            Student student = studentDAO.getStudentById(id);
    if (student == null) {            System.out.println("✗ Student not found.");
        return;        }                System.out.println("\nStudent to delete:");
    displayStudentDetails(student);
    System.out.print("\nAre you sure? (yes/no): ");
    String confirm = scanner.nextLine();
    if (confirm.equalsIgnoreCase("yes")) {            if (studentDAO.deleteStudent(id)) {
            System.out.println("✓ Student deleted successfully!");            } else {
            System.out.println("✗ Failed to delete student.");            }
    } else {            System.out.println("Deletion cancelled.");        }    }
        public void viewStatistics() {
    System.out.println("\n╔═══════════════════════════════════════╗");
    System.out.println("║           SYSTEM STATISTICS           ║");
    System.out.println("╚═══════════════════════════════════════╝");
    int totalStudents = studentDAO.getStudentCount();
    double avgGpa = studentDAO.getAverageGPA();
    int honorRollCount = studentDAO.getHonorRollStudents(3.5).size();
    System.out.println("Total Students: " + totalStudents);
    System.out.printf("Average GPA: %.2f\n", avgGpa);
    System.out.println("Honor Roll Students: " + honorRollCount);
    if (totalStudents > 0) {
        double honorPercentage = (honorRollCount * 100.0) / totalStudents;
        System.out.printf("Honor Roll Percentage: %.1f%%\n", honorPercentage);        }
}                private void displayStudentDetails(Student student) {
    System.out.println("─".repeat(40));
    System.out.println("Student ID: " + student.getStudentId());
    System.out.println("Name: " + student.getName());
    System.out.println("Age: " + student.getAge());
    System.out.println("Email: " + student.getEmail());
    System.out.println("Major: " + student.getMajor());
    System.out.printf("GPA: %.2f\n", student.getGpa());
    System.out.println("─".repeat(40));    }                public void run() {
    System.out.println("╔═══════════════════════════════════╗");
    System.out.println("║  Welcome to Student Management System  ║");
    System.out.println("║          Connected to PostgreSQL        ║");
    System.out.println("╚═══════════════════════════════════╝");
    // Test database connection        try {
        DatabaseConfig.getConnection().close();
        System.out.println("✓ Database connection verified!\n");
    } catch (Exception e) {
        System.out.println("✗ Database connection failed!");
        System.out.println("Please check your database configuration.\n");
        return;        }                int choice;        do {            displayMenu();
                    try {                choice = scanner.nextInt();
            scanner.nextLine();                                    switch (choice) {
                case 1: addStudent(); break;
                case 2: viewAllStudents(); break;
                case 3: findStudentById(); break;
                case 4: findStudentsByMajor(); break;
                case 5: viewHonorRoll(); break;
```

```
                    case 6: updateGPA(); break;
                    case 7: updateStudentInfo(); break;
                    case 8: deleteStudent(); break;
                    case 9: viewStatistics(); break;                    case 10:
                        System.out.println("\nThank you for using the system!");
                        System.out.println("Goodbye!");                        break;
                    default:
                        System.out.println("✗ Invalid choice! Please enter 1-10.");
                }                           } catch (Exception e) {
                System.out.println("✗ Invalid input!");                scanner.nextLine();
                choice = 0;            }                    } while (choice ≠ 10);
        scanner.close();    }                public static void main(String[] args) {
        StudentManagementApp app = new StudentManagementApp();        app.run();    }
}
```

📝 Homework Assignment

Task 1: Add Course Management (Required)

Create DAO for courses table:

1. Create `Course` model class with fields: courseId, courseName, credits, departmentId, teacherId
2. Create `CourseDAO` interface with CRUD methods
3. Implement `CourseDAOImpl` class
4. Add course management to main menu

Task 2: Implement Search Enhancement (Required)

Add advanced search capabilities:

1. Search students by name (partial match using LIKE)
2. Search students by GPA range (min and max)
3. Search students by age range
4. Combined search (major AND GPA range)

Task 3: Add Transaction Management (Challenge)

Implement enrollment system with transactions:

1. Create `Enrollment` model and DAO
2. Enroll student in course (must verify both exist)
3. Use transactions to ensure atomicity
4. Handle constraint violations properly

🎯 Assessment Checklist

Students should be able to:
- [ ] Understand JDBC architecture and components
- [ ] Add PostgreSQL JDBC driver to Java project
- [ ] Establish database connections using DriverManager
- [ ] Use PreparedStatement for safe SQL execution
- [ ] Execute SELECT queries and process ResultSet
- [ ] Perform INSERT, UPDATE, DELETE operations with executeUpdate()
- [ ] Implement DAO pattern for clean architecture
- [ ] Handle SQLExceptions properly
- [ ] Close resources correctly (Connection, Statement, ResultSet)
- [ ] Prevent SQL injection attacks
- [ ] Build complete data access layer
- [ ] Create model classes for database entities

🔧 Common Student Errors & Solutions

Error 1: ClassNotFoundException

```
java.lang.ClassNotFoundException: org.postgresql.Driver
```

Solution: Add PostgreSQL JDBC JAR to project classpath

Error 2: Connection Refused

```
org.postgresql.util.PSQLException: Connection refused
```

Solution:
- Check if PostgreSQL is running
- Verify host and port (default: localhost:5432)
- Check firewall settings

Error 3: Authentication Failed

```
FATAL: password authentication failed for user "postgres"
```

Solution: Check username and password in DatabaseConfig

Error 4: Database Does Not Exist

```
FATAL: database "university_db" does not exist
```

Solution: Create database in pgAdmin first

Error 5: Wrong Parameter Index

```
pstmt.setString(0, value);  // WRONG! Index starts at 1
pstmt.setString(1, value);  // CORRECT
```

Error 6: Resource Leak

```
Connection conn = DatabaseConfig.getConnection();
// ... use connection ...
// Forgot to close! Memory leak!
```

Solution: Always close in finally block

Error 7: Wrong Execute Method

```
// WRONG: executeQuery() returns nothing for INSERT
pstmt.executeQuery("INSERT INTO ... ");

// CORRECT: use executeUpdate() for INSERT/UPDATE/DELETE
pstmt.executeUpdate();
```

📚 Additional Resources

Official Documentation:

- JDBC Tutorial (Oracle)
- PostgreSQL JDBC Documentation
- PreparedStatement Guide

Best Practices:

- DAO Pattern Explained
- SQL Injection Prevention
- JDBC Best Practices

Video Tutorials:

- Search YouTube for "JDBC Tutorial"
- "Java Database Connectivity Explained"
- "PreparedStatement vs Statement"

🔜 Preview of Week 8

Next week is **Mid-Course Project Week**:
- **Project:** Complete Library Management System
- **Features:** Books, members, borrowings, due dates, fines
- **Technologies:** OOP + Collections + Exceptions + JDBC
- **Goal:** Consolidate all learning before Spring Boot

Requirements:
- Design complete database schema
- Implement all CRUD operations
- Add search and filter features
- Generate reports
- Handle all errors gracefully

Prepare: Review Weeks 1-7 materials!

📌 Key Takeaways

> ***"JDBC = Java ↔ Database Bridge"***
> *Connect your Java applications to any relational database*

> ***"Always Use PreparedStatement"***
> *Prevents SQL injection and improves performance*

> ***"DAO Pattern = Clean Architecture"***
> *Separates business logic from data access*

> ***"Close Your Resources"***
> *Prevents memory leaks and connection exhaustion*

> ***"executeQuery() for SELECT"***
> ***executeUpdate() for INSERT/UPDATE/DELETE"***

> ***"Index Starts at 1"***
> *PreparedStatement parameters begin at 1, not 0*

🎓 Week 7 Summary

Students have learned:
1. ✅ JDBC architecture and how it works
2. ✅ Setting up PostgreSQL JDBC driver
3. ✅ Establishing and managing database connections
4. ✅ Using PreparedStatement for safe SQL execution
5. ✅ Processing ResultSet data
6. ✅ Implementing full CRUD operations
7. ✅ DAO pattern for clean code organization
8. ✅ Exception handling for database operations
9. ✅ Resource management best practices
10. ✅ Building a complete data access layer

Major Achievement: Students now have a fully functional Student Management System with real database persistence! This is a significant milestone—they've built a professional-grade application using proper software architecture patterns.

Next Week: Time to put everything together in the mid-course project!1, studentId);

```
        // Execute query
        rs = pstmt.executeQuery();

        // Check if student exists
        if (rs.next()) {
            System.out.println("\n═══ Student Found ═══");
            System.out.println("ID: " + rs.getString("student_id"));
```

```
System.out.println("Age: " + rs.getInt("age"));
            System.out.println("Email: " + rs.getString("email"));
            System.out.println("Major: " + rs.getString("major"));
            System.out.println("GPA: " + rs.getDouble("gpa"));
        } else {
            System.out.println("✗ Student not found with ID: " + studentId);
        }

    } catch (SQLException e) {
        System.out.println("✗ Error: " + e.getMessage());
    } finally {
        DatabaseConfig.closeResources(connection, pstmt, rs);
    }
}

// Find students by major
public static void findStudentsByMajor(String major) {
    Connection connection = null;
    PreparedStatement pstmt = null;
    ResultSet rs = null;

    try {
        connection = DatabaseConfig.getConnection();

        String sql = "SELECT name, gpa FROM students WHERE major = ? ORDER BY gpa DESC";
        pstmt = connection.prepareStatement(sql);
        pstmt.setString(1, major);

        rs = pstmt.executeQuery();

        System.out.println("\n═══ Students in " + major + " ═══");
        int count = 0;
        while (rs.next()) {
            count++;
            System.out.printf("%d. %s (GPA: %.2f)\n",
                count, rs.getString("name"), rs.getDouble("gpa"));
        }

        if (count == 0) {
            System.out.println("No students found in this major.");
        } else {
            System.out.println("\nTotal: " + count + " students");
        }

    } catch (SQLException e) {
        System.out.println("✗ Error: " + e.getMessage());
    } finally {
        DatabaseConfig.closeResources(connection, pstmt, rs);
    }
}

// Find honor roll students (GPA ≥ threshold)
```

```
public static void findHonorRollStudents(double minGpa) {
    Connection connection = null;
    PreparedStatement pstmt = null;
    ResultSet rs = null;

    try {
        connection = DatabaseConfig.getConnection();

        String sql = "SELECT name, major, gpa FROM students WHERE gpa ≥ ? ORDER BY gpa DESC";
        pstmt = connection.prepareStatement(sql);
        pstmt.setDouble(1, minGpa);

        rs = pstmt.executeQuery();

        System.out.println("\n═══ Honor Roll (GPA ≥ " + minGpa + ") ═══");
        while (rs.next()) {
            System.out.printf("%s (%s) - GPA: %.2f\n",
                rs.getString("name"),
                rs.getString("major"),
                rs.getDouble("gpa"));
        }

    } catch (SQLException e) {
        System.out.println("✗ Error: " + e.getMessage());
    } finally {
        DatabaseConfig.closeResources(connection, pstmt, rs);
    }
}

public static void main(String[] args) {
    // Test different SELECT operations
    findStudentById("S001");
    findStudentsByMajor("Computer Science");
    findHonorRollStudents(3.5);
}
```

}

```
---

### Part 5: INSERT, UPDATE, DELETE Operations (15 minutes)

#### INSERT - Adding Records (5 minutes)

**Example 3: Inserting a Student**

```java
public class InsertStudent {
```
```

```java
public static boolean addStudent(String id, String name, int age,
                                 String email, String major, double gpa) {
        Connection connection = null;
        PreparedStatement pstmt = null;

        try {
            connection = DatabaseConfig.getConnection();

            // SQL INSERT statement with parameters
            String sql = "INSERT INTO students (student_id, name, age, email, major, gpa) " +
                         "VALUES (?, ?, ?, ?, ?, ?)";

            pstmt = connection.prepareStatement(sql);

            // Set all parameters
            pstmt.setString(1, id);
            pstmt.setString(2, name);
            pstmt.setInt(3, age);
            pstmt.setString(4, email);
            pstmt.setString(5, major);
            pstmt.setDouble(6, gpa);

            // Execute INSERT (use executeUpdate, not executeQuery!)
            int rowsAffected = pstmt.executeUpdate();

            if (rowsAffected > 0) {
                System.out.println("✓ Student added successfully!");
                return true;
            } else {
                System.out.println("✗ Failed to add student.");
                return false;
            }

        } catch (SQLException e) {
            // Handle specific errors
            if (e.getMessage().contains("duplicate key")) {
                System.out.println("✗ Error: Student ID already exists!");
            } else if (e.getMessage().contains("violates check constraint")) {
                System.out.println("✗ Error: Invalid data (check GPA and age)!");
            } else {
                System.out.println("✗ Error: " + e.getMessage());
            }
            return false;
        } finally {
            DatabaseConfig.closeResources(connection, pstmt, null);
        }
    }

    public static void main(String[] args) {
        // Add a new student
        boolean success = addStudent(
```

```
"John Wick",
            25,
            "john@university.edu",
            "Computer Science",
            3.8
        );

        if (success) {
            System.out.println("Student added to database!");
        }
    }
}
```

Important Notes:
- Use `executeUpdate()` for INSERT, UPDATE, DELETE
- Returns number of rows affected
- Use `executeQuery()` ONLY for SELECT
- Always handle `SQLException`

UPDATE - Modifying Records (5 minutes)

Example 4: Updating Student Data

```java
public class UpdateStudent {

    // Update student's GPA
    public static boolean updateStudentGPA(String studentId, double newGpa) {
        Connection connection = null;
        PreparedStatement pstmt = null;

        try {
            connection = DatabaseConfig.getConnection();

            String sql = "UPDATE students SET gpa = ? WHERE student_id = ?";
            pstmt = connection.prepareStatement(sql);

            // Set parameters (order matters!)
            pstmt.setDouble(1, newGpa);
            pstmt.setString(2, studentId);

            int rowsAffected = pstmt.executeUpdate();

            if (rowsAffected > 0) {
                System.out.println("✓ GPA updated successfully!");
                return true;
            } else {
                System.out.println("✗ Student not found with ID: " + studentId);
                return false;
            }
```

```
                    } catch (SQLException e) {
            System.out.println("✗ Error updating GPA: " + e.getMessage());
            return false;        } finally {
            DatabaseConfig.closeResources(connection, pstmt, null);        }    }
    // Update multiple columns
    public static boolean updateStudentInfo(String studentId, String newEmail, String
         newMajor) {
        Connection connection = null;        PreparedStatement pstmt = null;
        try {            connection = DatabaseConfig.getConnection();
            String sql = "UPDATE students SET email = ?, major = ? WHERE student_id = ?";
            pstmt = connection.prepareStatement(sql);
            pstmt.setString(1, newEmail);            pstmt.setString(2, newMajor);
            pstmt.setString(3, studentId);
            int rowsAffected = pstmt.executeUpdate();
            if (rowsAffected > 0) {
                System.out.println("✓ Student information updated!");
                return true;            } else {
                System.out.println("✗ Student not found.");                return false;
            }                    } catch (SQLException e) {
            System.out.println("✗ Error: " + e.getMessage());            return false;
        } finally {            DatabaseConfig.closeResources(connection, pstmt, null);
        }    }                public static void main(String[] args) {
        // Test UPDATE operations        updateStudentGPA("S001", 3.9);
        updateStudentInfo("S001", "alice.new@university.edu", "Computer Engineering");
    }    }
```

⚠️ **Critical Warning:**

```
// DANGER: Updates ALL students!
String sql = "UPDATE students SET gpa = 4.0";  // Missing WHERE clause!

// CORRECT: Update specific student
String sql = "UPDATE students SET gpa = 4.0 WHERE student_id = ?";
```

Always use WHERE clause unless you intend to update all rows!

DELETE - Removing Records (5 minutes)

Example 5: Deleting Records

```
public class DeleteStudent {

    public static boolean deleteStudent(String studentId) {
        Connection connection = null;
        PreparedStatement pstmt = null;
        ResultSet rs = null;

        try {
            connection = DatabaseConfig.getConnection();
```

```
                        // First, check if student exists and get name
            String checkSql = "SELECT name FROM students WHERE student_id = ?";
            pstmt = connection.prepareStatement(checkSql);
            pstmt.setString(1, studentId);            rs = pstmt.executeQuery();
            if (!rs.next()) {
                System.out.println("✗ Student not found with ID: " + studentId);
                return false;            }
            String studentName = rs.getString("name");            rs.close();
            pstmt.close();                        // Now delete the student
            String deleteSql = "DELETE FROM students WHERE student_id = ?";
            pstmt = connection.prepareStatement(deleteSql);
            pstmt.setString(1, studentId);
            int rowsAffected = pstmt.executeUpdate();
            if (rowsAffected > 0) {
                System.out.println("✓ Student '" + studentName + "' deleted successfully!");
                return true;            } else {                return false;            }
                    } catch (SQLException e) {
            // Handle foreign key constraint errors
            if (e.getMessage().contains("foreign key")) {
                System.out.println("✗ Cannot delete: Student has enrollments in courses!");
                System.out.println("  Remove enrollments first, or use CASCADE.");
            } else {                System.out.println("✗ Error: " + e.getMessage());
            }            return false;        } finally {
            DatabaseConfig.closeResources(connection, pstmt, rs);        }    }
    // Delete students by condition (be careful!)
    public static int deleteStudentsByGPA(double maxGpa) {
        Connection connection = null;        PreparedStatement pstmt = null;
        try {            connection = DatabaseConfig.getConnection();
            String sql = "DELETE FROM students WHERE gpa < ?";
            pstmt = connection.prepareStatement(sql);            pstmt.setDouble(1, maxGpa);
                        int rowsDeleted = pstmt.executeUpdate();
            System.out.println("✓ Deleted " + rowsDeleted + " students with GPA < " + maxGpa);
            return rowsDeleted;                    } catch (SQLException e) {
            System.out.println("✗ Error: " + e.getMessage());            return 0;
        } finally {            DatabaseConfig.closeResources(connection, pstmt, null);
        }    }                public static void main(String[] args) {
        // Delete a specific student        deleteStudent("S010");
        // Be VERY careful with this type of deletion!
        // deleteStudentsByGPA(2.0);  // Deletes all students with GPA < 2.0    }    }
```

⚠️ Critical Warning:

```
// DANGER: Deletes ALL students!
String sql = "DELETE FROM students";  // Missing WHERE clause!

// CORRECT: Delete specific student
String sql = "DELETE FROM students WHERE student_id = ?";
```

Part 6: Data Access Object (DAO) Pattern (10 minutes)

What is DAO Pattern? (3 minutes)

DAO = Data Access Object
- Design pattern that separates database logic from business logic
- Provides clean interface for CRUD operations
- Makes code more maintainable, testable, and reusable
- Follows Single Responsibility Principle

Architecture:

```
Application Layer
       ↓
Service/Business Logic Layer
       ↓
DAO Interface (Contract)
       ↓
DAO Implementation (Database operations)
       ↓
DatabaseConfig
       ↓
PostgreSQL Database
```

Benefits:
- ✅ Separation of concerns
- ✅ Easy to test (can mock DAO)
- ✅ Easy to switch databases
- ✅ Reusable code
- ✅ Clean architecture

Creating Student Model and DAO (7 minutes)

Step 1: Student Model Class

```java
public class Student {
    private String studentId;
    private String name;
    private int age;
    private String email;
    private String major;
    private double gpa;

    // Constructor with all fields
    public Student(String studentId, String name, int age, String email,
                   String major, double gpa) {
```

```
        this.studentId = studentId;        this.name = name;        this.age = age;
        this.email = email;        this.major = major;        this.gpa = gpa;    }
    // Getters    public String getStudentId() { return studentId; }
    public String getName() { return name; }    public int getAge() { return age; }
    public String getEmail() { return email; }    public String getMajor() { return major; }
    public double getGpa() { return gpa; }            // Setters
    public void setStudentId(String studentId) { this.studentId = studentId; }
    public void setName(String name) { this.name = name; }
    public void setAge(int age) { this.age = age; }
    public void setEmail(String email) { this.email = email; }
    public void setMajor(String major) { this.major = major; }
    public void setGpa(double gpa) { this.gpa = gpa; }
    // toString for easy printing    @Override    public String toString() {
        return String.format("Student[ID=%s, Name=%s, Age=%d, Major=%s, GPA=%.2f]",
            studentId, name, age, major, gpa);    }    }
```

Step 2: DAO Interface (Contract)

```java
import java.util.List;

public interface StudentDAO {
    // Create
    boolean addStudent(Student student);

    // Read
    Student getStudentById(String studentId);
    List<Student> getAllStudents();
    List<Student> getStudentsByMajor(String major);
    List<Student> getHonorRollStudents(double minGpa);

    // Update
    boolean updateStudent(Student student);
    boolean updateGPA(String studentId, double newGpa);

    // Delete
    boolean deleteStudent(String studentId);

    // Statistics
    int getStudentCount();
    double getAverageGPA();
}
```

Step 3: DAO Implementation (Partial - will complete in hands-on)

```java
import java.sql.*;
import java.util.ArrayList;
import java.util.List;
```

public class StudentDAOImpl implements StudentDAO {

```
@Override
public boolean addStudent(Student student) {
    Connection conn = null;
    PreparedStatement pstmt = null;

    try {
        conn = DatabaseConfig.getConnection();
        String sql = "INSERT INTO students (student_id, name, age, email, major, gpa) " +
                     "VALUES (?, ?, ?, ?, ?, ?)";

        pstmt = conn.prepareStatement(sql);
        pstmt.setString(1, student.getStudentId());
        pstmt.setString(2, student.getName());
        pstmt.setInt(3, student.getAge());
        pstmt.setString(4, student.getEmail());
        pstmt.setString(
```